In [18]:
```
from huggingface_hub import login
login()
```

```
VBox(children=(HTML(value='<center> <img\nsrc=https://huggingface.co/front/assets/huggingface_logo-noborder.sv…
```

In [11]:
```
# Install necessary libraries
!pip install -q torch transformers sentence-transformers faiss-cpu PyPDF2 hf_xet gradio
```

In [1]:
```
from PyPDF2 import PdfReader

def extract_text_from_pdf(pdf_path):
    reader = PdfReader(pdf_path)
    text = ""
    for page in reader.pages:
        text += page.extract_text()
    return text

# Upload a PDF file
from google.colab import files
uploaded = files.upload()

# Extract text from the uploaded PDF
pdf_path = list(uploaded.keys())[0]
document_text = extract_text_from_pdf(pdf_path)
print(f"Extracted text length: {len(document_text)} characters")
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Md_Zaid_Hussain_Artificial_Intelligence_Engineer_Resume.pdf to Md_Zaid_Hussain_Artificial_Intelligence_Engineer_Resume (1).pdf
Extracted text length: 7388 characters
```

In [19]:
```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

def split_text_into_chunks(text, max_length=512):
    tokens = tokenizer.tokenize(text)
    chunks = []
    for i in range(0, len(tokens), max_length):
        chunk = tokens[i:i + max_length]
        chunks.append(tokenizer.convert_tokens_to_string(chunk))
    return chunks

chunks = split_text_into_chunks(document_text)
print(f"Number of chunks: {len(chunks)}")
```

```
Token indices sequence length is longer than the specified maximum sequence length for this model (1555 > 512). Running this sequence through the model will result in indexing errors
```

```
Number of chunks: 4
```

In [3]:
```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')

chunk_embeddings = model.encode(chunks, convert_to_tensor=True)
print(f"Embeddings shape: {chunk_embeddings.shape}")
```

```
Embeddings shape: torch.Size([4, 384])
```

In [4]:
```
import faiss
import numpy as np

# Convert embeddings to numpy array
chunk_embeddings_np = chunk_embeddings.cpu().numpy()

# Build FAISS index
index = faiss.IndexFlatL2(chunk_embeddings_np.shape[1])  # L2 distance
index.add(chunk_embeddings_np)

print(f"FAISS index size: {index.ntotal}")
```

```
FAISS index size: 4
```

In [5]:
```
def retrieve_relevant_chunks(query, top_k=3):
    query_embedding = model.encode(query, convert_to_tensor=True)
    query_embedding_np = query_embedding.cpu().numpy()
```

```
    # Perform similarity search
    distances, indices = index.search(np.array([query_embedding_np]), top_k)

    relevant_chunks = [chunks[i] for i in indices[0]]
    return relevant_chunks

# Example query
query = "What is the main topic of the document?"
relevant_chunks = retrieve_relevant_chunks(query)
print("Relevant chunks:\n", "\n---\n".join(relevant_chunks))
```

```
Relevant chunks:
 jan 202 2 languages • english ( fluent ), hindi ( fluent ), spanish ( intermediate )
---
mohammed zaid hussain artificial intelligence engineer + 91 93302 77380 | email linkedin | kolkata, west bengal, india github profile summary artifi
---
learn • soft skills • stakeholder communication : translated technical ai concepts for non - technical executives ( presented hybrid ai roulette pre
```

In [14]:

```
from transformers import pipeline, AutoTokenizer

# Load a generative model and tokenizer
model_name = "google/flan-t5-large"
tokenizer = AutoTokenizer.from_pretrained(model_name)
generator = pipeline("text2text-generation", model=model_name, device=0)  # Use GPU if available

def generate_answer(query, relevant_chunks):
    # Combine relevant chunks into a single context
    context = " ".join(relevant_chunks)

    # Truncate the context to fit within the model's max token limit
    truncated_context = tokenizer.decode(
        tokenizer.encode(context, truncation=True, max_length=512),
        skip_special_tokens=True
    )

    # Create a refined prompt
    prompt = (
        f"Context: {truncated_context}\n"
        f"Question: {query}\n"
        "Instructions: Carefully analyze the context and provide a clear, concise answer. "
        "If the question asks for a list or count, ensure all items are included.\n"
        "Answer:"
    )

    # Generate the answer
    answer = generator(prompt, max_length=150)[0]['generated_text']
    return answer
```

```
Device set to use cuda:0
```

In [17]:

```
import gradio as gr
from PyPDF2 import PdfReader
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
from transformers import pipeline, AutoTokenizer

# Step 1: Extract text from PDF
def extract_text_from_pdf(pdf_path):
    reader = PdfReader(pdf_path)
    text = ""
    for page in reader.pages:
        text += page.extract_text()
    return text

# Step 2: Split text into chunks
def split_text_into_chunks(text, max_length=512):
    tokens = text.split()
    chunks = []
    for i in range(0, len(tokens), max_length):
        chunk = " ".join(tokens[i:i + max_length])
        chunks.append(chunk)
    return chunks

# Step 3: Generate embeddings and build FAISS index
def build_faiss_index(chunks):
    embedding_model = SentenceTransformer('all-MiniLM-L6-v2')
    embeddings = embedding_model.encode(chunks, convert_to_tensor=True)
    embeddings_np = embeddings.cpu().numpy()
```

```
    index = faiss.IndexFlatL2(embeddings_np.shape[1])
    index.add(embeddings_np)
    return index, embedding_model

# Step 4: Retrieve relevant chunks
def retrieve_relevant_chunks(query, index, embedding_model, chunks, top_k=3):
    query_embedding = embedding_model.encode(query, convert_to_tensor=True)
    query_embedding_np = query_embedding.cpu().numpy()
    distances, indices = index.search(np.array([query_embedding_np]), top_k)
    relevant_chunks = [chunks[i] for i in indices[0]]
    return relevant_chunks

# Step 5: Generate answer using a generative model
model_name = "google/flan-t5-large"
tokenizer = AutoTokenizer.from_pretrained(model_name)
generator = pipeline("text2text-generation", model=model_name, device=0)

def generate_answer(query, relevant_chunks):
    # Combine relevant chunks into a single context
    context = " ".join(relevant_chunks)

    # Truncate the context to fit within the model's max token limit
    truncated_context = tokenizer.decode(
        tokenizer.encode(context, truncation=True, max_length=512),
        skip_special_tokens=True
    )

    # Create a refined prompt
    prompt = (
        f"Context: {truncated_context}\n"
        f"Question: {query}\n"
        "Instructions: Carefully analyze the context and provide a clear, concise answer. "
        "If the question asks for a list or count, ensure all items are included.\n"
        "Answer:"
    )

    # Generate the answer
    answer = generator(prompt, max_length=150)[0]['generated_text']
    return answer

# Step 6: Main RAG system function
def rag_system(pdf_file, query):
    # Step 1: Extract text from the uploaded PDF
    pdf_path = pdf_file.name
    document_text = extract_text_from_pdf(pdf_path)

    # Step 2: Split text into chunks
    chunks = split_text_into_chunks(document_text)

    # Step 3: Build FAISS index
    index, embedding_model = build_faiss_index(chunks)

    # Step 4: Retrieve relevant chunks
    relevant_chunks = retrieve_relevant_chunks(query, index, embedding_model, chunks)

    # Step 5: Generate answer
    answer = generate_answer(query, relevant_chunks)
    return answer

# Step 7: Create Gradio interface
with gr.Blocks() as demo:
    gr.Markdown("# PDF-Based Question Answering System")
    gr.Markdown("Upload a PDF and ask questions based on its content.")

    with gr.Row():
        pdf_input = gr.File(label="Upload PDF")
        question_input = gr.Textbox(label="Enter Your Question")
        answer_output = gr.Textbox(label="Generated Answer")

    submit_button = gr.Button("Submit")

    submit_button.click(
        rag_system,
        inputs=[pdf_input, question_input],
        outputs=answer_output
    )
```

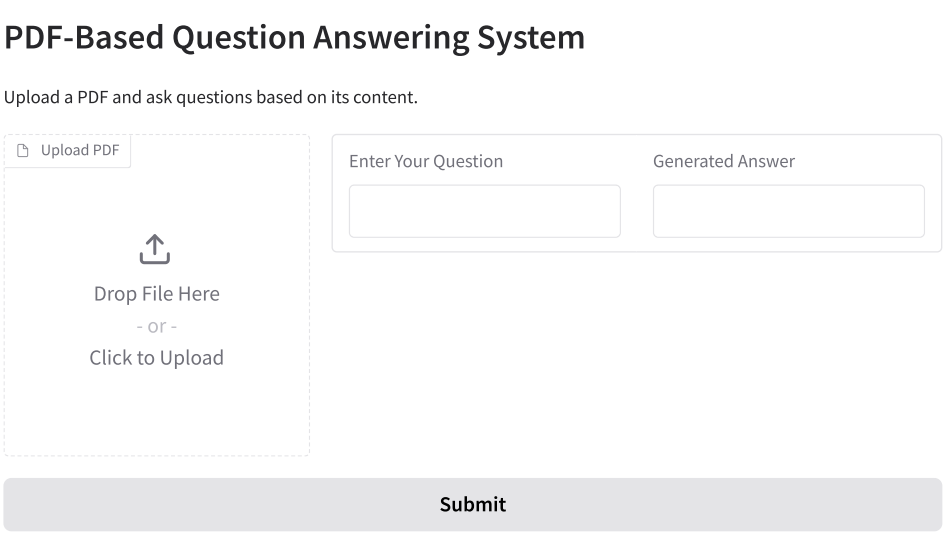
```
# Launch the Gradio app
demo.launch()
```

```
Device set to use cuda
```

```
It looks like you are running Gradio on a hosted a Jupyter notebook. For the Gradio app to work, sharing must be enabled. Automatically setting `sha

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://d64553f5348f1f7451.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to de
```



Out [17]:

In [ ]: